

Regime-Aware Machine Learning for Technical Pattern Trading

Event-Based Classification and Profitability Evaluation
on S&P 500 ETF (SPY) Daily Data

Zeineb Turki

Supervisor: [Supervisor Name]

[University Name]

[Course / Program Name]

Summer Semester 2026

May 2026

Assignment Statement

[Placeholder: Insert the official signed assignment/task sheet here. This page should contain the formal task description as provided by the supervisor or university administration.]

Table of Contents

1. Abstract
 2. Introduction
 3. Background and Related Work
 4. System Design and Design Decisions
 5. Data and Preprocessing
 6. Pattern Detection Methodology
 7. Feature Engineering
 8. Triple-Barrier Labeling
 9. Machine Learning Models
 10. Validation Methodology
 11. Trading Simulation and Profitability Evaluation
 12. Hyperparameter Optimization
 13. Experimental Results
 14. Generalization and Variance Analysis
 15. F-Beta Analysis
 16. Discussion
 17. Limitations
 18. Future Work
 19. Conclusion
 20. Bibliography
- Appendix A: Module Overview
- Appendix B: Parameter Reference
- Appendix C: Notebook Guide

1. Abstract

This report presents a regime-aware machine learning system for trading S&P 500 ETF (SPY) daily data. The system detects technical patterns (support/resistance, channels, triangles, and multiple tops/bottoms), extracts 48 leakage-free features per event, and assigns labels using triple-barrier labeling. Random Forest and Bagging classifiers are trained on 132 detected events across 4,023 trading days (2010–2025). A key contribution is treating the triple-barrier parameters (profit target, stop loss, and maximum holding period) as tunable hyperparameters, optimized via grid search across 100 configurations. The system evaluates models by both classification metrics (F1 macro: 0.569, 3.6× over baseline) and trading profitability (cumulative return: 25.9%). A central finding is that optimal parameters for classification differ from those for profitability, confirming that accuracy alone is insufficient for evaluating trading models. Touch-based events augment the dataset by 36.5% (38 additional events). All results are preliminary given the small dataset size and simplified trading assumptions.

2. Introduction

2.1 Motivation

Financial markets present one of the most challenging domains for machine learning. Price series are extremely noisy, non-stationary, and influenced by regime shifts, macroeconomic events, and structural market changes. Most individual price bars do not contain a repeatable, actionable signal. Consequently, models trained to predict every bar's direction typically learn to exploit price level trends or temporal patterns rather than genuine trading opportunities.

This project takes an alternative approach: **event-based learning**. Instead of predicting every bar, the system first identifies technically meaningful moments—bars where price interacts with established chart patterns—and then predicts the outcome of a trade-sized move initiated at these moments. This reduces the problem from thousands of noisy bars to a focused set of candidate trading events.

2.2 Research Questions

The project addresses three main questions:

1. Can technical pattern detection combined with machine learning classify trade outcomes better than a random baseline?
2. Do the optimal labeling parameters for classification accuracy also optimize trading profitability?
3. Does augmenting the event set with touch-based signals improve model performance?

2.3 Main Contributions

The project makes the following contributions:

- An end-to-end pipeline from raw OHLCV data to profitability-evaluated predictions, implemented as modular Python code.
- Four complementary pattern detectors with signal localization and cooldown filtering.
- 48 leakage-free features combining technical indicators, pattern geometry, and event-type encodings.
- Conversion of triple-barrier labeling parameters from fixed constants to tunable hyperparameters, with grid search optimization targeting both classification and profitability.
- A trading simulation module that evaluates models by cumulative return, Sharpe ratio, win rate, profit factor, and maximum drawdown.
- Touch-based event augmentation that increases the dataset by 36.5%.

3. Background and Related Work

3.1 Technical Analysis

Technical analysis is the study of past price and volume data to forecast future price movements. While its theoretical foundations are debated, specific chart patterns are widely used by practitioners and have been formalized in academic literature (Lo, Mamaysky & Wang, 2000). The patterns used in this project—support/resistance levels, price channels, triangle formations, and multiple tops/bottoms—are among the most commonly cited in both trading practice and quantitative finance research.

3.2 Key Technical Indicators

Average True Range (ATR): A volatility measure computed as the rolling mean of True Range (the maximum of: $\text{High} - \text{Low}$, $|\text{High} - \text{prev Close}|$, $|\text{Low} - \text{prev Close}|$). ATR is used throughout this project as a scale-independent distance measure for barriers, proximity thresholds, and channel widths.

Relative Strength Index (RSI): A momentum oscillator bounded between 0 and 100, measuring the speed and magnitude of recent price changes. Values above 70 indicate overbought conditions; below 30 indicates oversold.

MACD (Moving Average Convergence Divergence): The difference between fast and slow exponential moving averages. In this project, MACD values are normalized by dividing by the Close price to prevent scale leakage across different price regimes.

Bollinger Bands: A volatility envelope defined as a simple moving average ± 2 standard deviations. The width and the position of price within the bands (%B) are used as features.

3.3 Triple-Barrier Labeling

The triple-barrier method, introduced by Lopez de Prado (2018) in *Advances in Financial Machine Learning*, assigns labels based on which of three barriers is touched first after a trading signal:

- **Upper barrier (profit target):** $\text{entry price} + \text{pt_mult} \times \text{ATR}$
- **Lower barrier (stop loss):** $\text{entry price} - \text{sl_mult} \times \text{ATR}$
- **Time barrier:** maximum holding period in bars

This approach labels events according to realistic trade outcomes rather than arbitrary future price direction, making it well-suited for trading applications.

3.4 Random Forest and Bagging

Random Forest (Breiman, 2001) constructs an ensemble of decision trees, each trained on a bootstrap sample with random feature subsets at each split. Bagging uses the same bootstrap aggregation but without feature subsetting. Both are robust to noise, handle mixed feature types naturally, and provide feature importance rankings. Balanced class weighting adjusts for label imbalance.

3.5 Walk-Forward Validation

Walk-forward cross-validation simulates realistic model deployment by training on historical data and testing on subsequent unseen periods. Unlike standard k-fold CV, it respects temporal ordering and prevents future information from leaking into training. This is essential in financial applications

where time-series structure invalidates the i.i.d. assumption.

3.6 Related Work in Financial Machine Learning

The application of machine learning to financial time series has grown significantly since the 2010s. Krauss, Do & Huck (2017) provided a comprehensive comparison of deep learning, gradient boosting, and random forests for statistical arbitrage, finding that ensemble methods consistently outperform single models on S&P 500 data. Fischer & Krauss (2018) demonstrated that LSTM networks could capture temporal dependencies in financial data, though they noted significant regime-dependence in performance.

Closer to our work, Sezer, Gudelek & Ozbayoglu (2020) surveyed deep learning applications in financial time series, identifying that most approaches fail to account for transaction costs and regime shifts. Patel et al. (2015) compared SVMs, Random Forests, and neural networks for stock prediction, finding that technical indicator features outperformed raw price features across all models.

The triple-barrier labeling method we employ originates from Lopez de Prado (2018), whose work on financial machine learning introduced several innovations: trade-outcome-based labeling, purged cross-validation, and meta-labeling. Our project adopts the triple-barrier framework but extends it by treating the barrier parameters as tunable hyperparameters rather than fixed constants.

Bailey et al. (2014) warned about the dangers of backtest overfitting — the tendency for strategy selection to identify spuriously profitable configurations. This concern directly motivates our use of walk-forward CV and our honest reporting of variance across folds.

4. System Design and Design Decisions

4.1 Architecture Overview

The system follows a modular pipeline architecture, implemented as a Python package with clearly separated concerns:

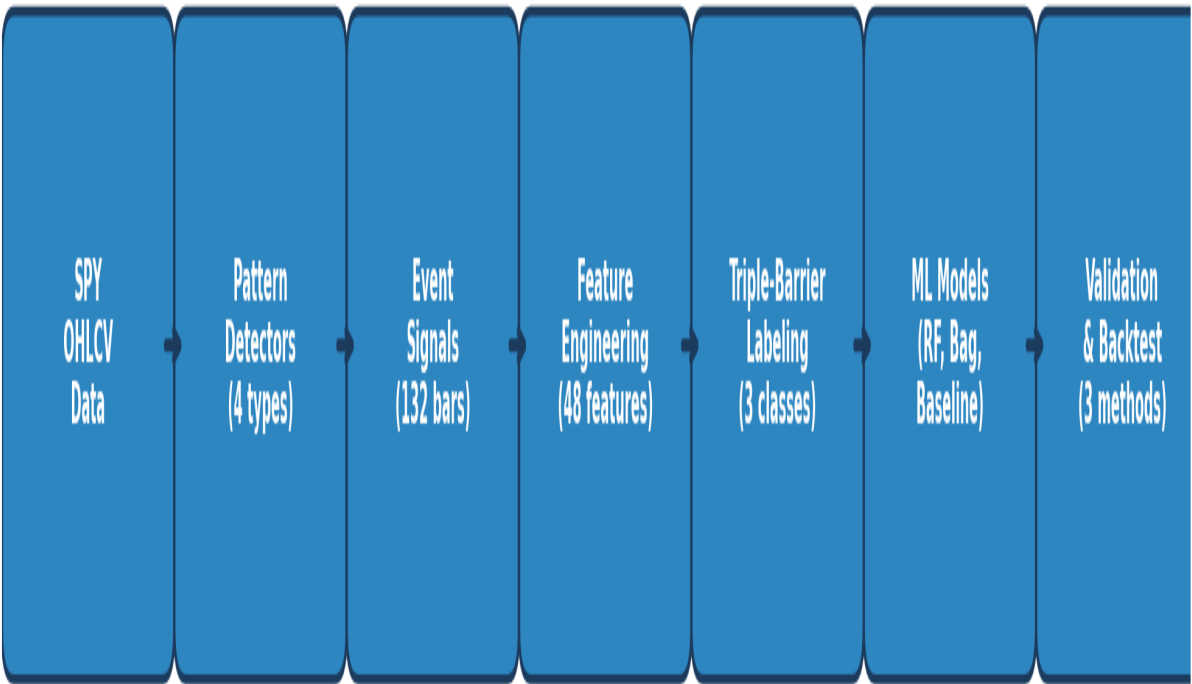


Figure 1: System architecture — end-to-end pipeline from raw data to profitability evaluation.

4.2 Module Structure

Module	Key Files	Responsibility
src/data/	load_data.py, utils.py	Data loading (CSV, yfinance, Alpha Vantage), ATR computation
src/patterns/	scanner.py, 5 detectors	Pattern detection, event scanning, touch events
src/labeling/	label_events.py	Triple-barrier labeling with configurable parameters
src/features/	indicators.py, build_features.py	48 technical features, geometry, event dummies
src/models/	train.py, optimize.py	RF/Bagging training, CV, hyperparameter optimization
src/backtest/	simulator.py	Trade simulation, profitability metrics
notebooks/	12 Jupyter notebooks	Interactive analysis, visualization, experiments
reports/	11 generators	PDF report and thesis generation

Table 1: Module structure and responsibilities.

4.3 Design Principles

- **No lookahead leakage:** every feature at bar i uses only data up to and including bar i .
- **Chronological splitting:** training always precedes validation and test data in time.
- **Modular independence:** each module can be used standalone or as part of the pipeline.
- **Configurable parameters:** all thresholds are exposed as function arguments with documented defaults.

4.4 Key Design Decisions

Why Random Forest and Bagging? Tree-based ensembles were chosen over deep learning methods for several reasons: (1) the dataset is small (~100 events), making neural networks prone to severe overfitting; (2) RF handles mixed feature types natively without preprocessing; (3) feature importance rankings provide interpretability; (4) ensemble diversity through bootstrap sampling and random feature subsets provides natural regularisation.

Why event-based labeling? Training on every bar would expose the model to thousands of noisy, low-information observations. By restricting training to technically meaningful events, we focus the learning problem on moments where price structure may provide a genuine predictive edge. This also aligns the prediction task with a practical trading decision: *given this pattern, should I trade?*

Why TP/SL as hyperparameters? The triple-barrier parameters do not merely tune the model — they *redefine the classification task*. A trade with $pt=1.5$ and $sl=1.5$ targets a different risk/reward profile than $pt=2.5$ and $sl=3.0$. Treating them as fixed constants would embed an untested assumption about optimal risk management into the labels.

Why normalised indicators? Raw price-level features (ATR in dollars, SMA in dollars) would allow the model to distinguish early vs. late periods by magnitude alone, inflating apparent generalisation. All features are normalised: $ATR \rightarrow atr_ratio$ ($ATR/Close$), $SMA \rightarrow sma_dist$ ($(Close-SMA)/SMA$), $MACD \rightarrow macd_norm$ ($MACD/Close$).

Why touch-based events? The original detectors produce only ~100 events across 4,000 bars, limiting statistical power. Touch-based events add boundary-interaction signals with a tighter proximity threshold (0.2 vs 0.3 ATR), following the supervisor's guidance to explore sequences starting from direct trendline contact.

5. Data and Preprocessing

5.1 Dataset Description

The dataset consists of daily OHLCV (Open, High, Low, Close, Volume) data for the SPDR S&P 500 ETF Trust (SPY), covering 4,023 trading days from January 4, 2010 to December 30, 2025. SPY was chosen because it is the most liquid equity ETF, minimizing the impact of illiquidity on pattern detection.

Property	Value
Ticker	SPY
Period	2010-01-04 to 2025-12-30
Trading days	4,023
Price range	\$77.36 – \$690.38
Data sources	CSV cache, Yahoo Finance, Alpha Vantage
Required columns	Open, High, Low, Close, Volume

Table 2: Dataset characteristics.

5.2 Data Cleaning

The loading pipeline performs automatic cleaning: timezone removal from DatetimeIndex, chronological sorting, NaN removal, and validation that all required OHLCV columns are present. The primary data source is a local CSV cache; Yahoo Finance and Alpha Vantage serve as fallbacks.

5.3 Indicator Computation

All technical indicators are computed bar-by-bar using only current and historical data. The 14-period Average True Range (ATR) serves as the fundamental volatility measure, used for barrier distances, proximity thresholds, and normalization throughout the system.

6. Pattern Detection Methodology

The system employs four complementary pattern detectors, each targeting a different type of technically significant price structure. All detectors share common design principles: ATR-based thresholds for scale independence, cooldown filters (10 bars) to prevent signal clustering, and signal localization at the specific bar where the pattern condition is met.

6.1 Support and Resistance

Support and resistance levels are identified using rolling extremes of the High (resistance) and Low (support) prices over a 50-bar window (≈ 2.5 months). Two filters reduce false positives:

- **Level stability:** the level must be unchanged for at least 5 consecutive bars, filtering out continuously rising resistance in uptrends.
- **Proximity test:** Close must be within $0.3 \times \text{ATR}(14)$ of a stable level.

Signals are suppressed for 10 bars after each detection to avoid flagging the same approach as multiple events.

6.2 Channel Detection

Price channels are detected using the chunk-extremes method from the TrendLineChannelDetection reference. The algorithm:

1. Divides a dynamic lookback window (40 ± 15 bars) into non-overlapping 5-bar chunks.
2. Extracts the maximum-High and minimum-Low bar from each chunk.
3. Fits first-degree polynomials (polyfit) to the chunk extremes for upper and lower boundaries.
4. Adjusts intercepts so the upper line caps all chunk highs and the lower line floors all chunk lows.
5. Validates with swing-pivot touches (≥ 2 upper, ≥ 3 lower, within $0.20 \times \text{ATR}$), parallelism (slope difference ≤ 0.25), minimum width ($1-6 \times \text{ATR}$), and containment ($\geq 70\%$ of bars inside).

Confidence scores weight touches (40%), containment (25%), parallelism (20%), and width (15%), with a 20% penalty if no rejection confirmation at the boundary.

6.3 Triangle Detection

Triangles are identified from swing pivots (local extremes within ± 3 bars) using linear regression:

1. Find swing highs and swing lows within a 25-bar lookback window.
2. Fit linear regression (scipy.stats.linregress) to each set of pivots.
3. Require $|r| \geq 0.85$ on each trendline for tight pivot alignment.
4. Check convergence: at least 5% range compression between the trendlines.
5. Classify by slope: ascending (rising lower, flat upper), descending (falling upper, flat lower), or symmetric (both converging).
6. Fire breakout signal when price exceeds the recent 3-bar range by $0.3 \times \text{ATR}$.

Note: Per supervisor feedback, triangle and channel events are excluded from training labels due to concerns about detection accuracy. They are retained for pattern geometry features and discussed for future improvement.

6.4 Multiple Tops and Bottoms

Multiple top/bottom patterns are detected using rolling extremes combined with close-trend confirmation:

- **Multiple top:** rolling 20-bar maximum of highs remains at its ceiling while the 5-bar close slope turns negative.
- **Multiple bottom:** rolling 20-bar minimum of lows remains at its floor while the 5-bar close slope turns positive.

6.5 Touch-Based Event Expansion

Following supervisor guidance to *"start sequences from direct touch of trend lines,"* the system generates additional events when price directly touches support/resistance levels or channel boundaries. Touch events use a tighter proximity threshold ($0.2 \times \text{ATR}$ vs. $0.3 \times \text{ATR}$ for standard detectors) and are tracked separately from original detector events.

Event Source	Count	Percentage
Standard detectors	104	73.2%
Touch-only events	38	26.8%
Combined total	142	100%

Table 3: Event counts by source.

6.6 Detection Summary

Detector	Events	Key Thresholds
Support / Resistance	42	window=50, $0.3 \times \text{ATR}$, stability=5 bars
Triangles	17	window=25, $ r \geq 0.85$, 5% convergence
Channels	12	lookback= 40 ± 15 , $\geq 2+3$ touches, $\geq 70\%$ containment
Multiple Tops/Bottoms	63	window=20, 5-bar slope confirmation
Touch events (additional)	38	$0.2 \times \text{ATR}$ proximity, cooldown=10
TOTAL (combined)	142	3.5% event density

Table 4: Pattern detection summary.

7. Feature Engineering

Each detected event is represented by 48 features extracted from three sources: bar-level technical indicators (computed at the event bar), pattern geometry metadata, and event-type encodings. All features use only information available at or before the event bar—no future data is used.

7.1 Feature Groups

Group	Count	Examples	Source
Volatility	2	atr_ratio (ATR/Close), rvol_20	indicators.py
Returns	4	ret_1, ret_5, ret_10, ret_20	indicators.py
Momentum (ROC)	3	mom_5, mom_10, mom_20	indicators.py
MA Distances	5	sma_10_dist ... sma_200_dist	indicators.py
MA Spreads	3	ma_spread_10_50, 20_200, 50_200	indicators.py
RSI	1	rsi_14	indicators.py
MACD (normalized)	3	macd_norm, signal_norm, hist_norm	indicators.py
Bollinger Bands	2	bb_width, bb_pctb	indicators.py
Volume	2	volume_ratio, volume_std	indicators.py
Binary Filters	8	BB touches, SMA crosses, RSI extremes	indicators.py
Pattern Geometry	11	slopes, touches, containment, width, R ²	build_features.py
Event Type Dummies	4–6	One-hot encoded pattern type	build_features.py
TOTAL	48–50		

Table 5: Feature groups in the final feature matrix.

7.2 Leakage Prevention

Several features that are common in technical analysis were deliberately excluded because they leak temporal information:

- **Raw ATR (atr_14):** scales with price level (SPY at \$100 vs \$600). Replaced with atr_ratio = ATR / Close.
- **Absolute SMA values:** trend upward with price over time, serving as time proxies. Replaced with relative distance features (sma_*_dist = (Close – SMA) / SMA).
- **Cumulative OBV:** grows monotonically over time. Replaced with volume_ratio and volume_std (rolling, bounded measures).
- **Raw MACD:** scales with price level. Replaced with price-normalized versions (macd_norm = MACD_line / Close).
- **Entry price and event ATR:** directly encode the price level at the event bar. Excluded from features; volatility regime is captured by atr_ratio instead.

8. Triple-Barrier Labeling

8.1 Method Description

The triple-barrier method assigns directional labels to each event based on which barrier is touched first when walking forward from the signal bar:

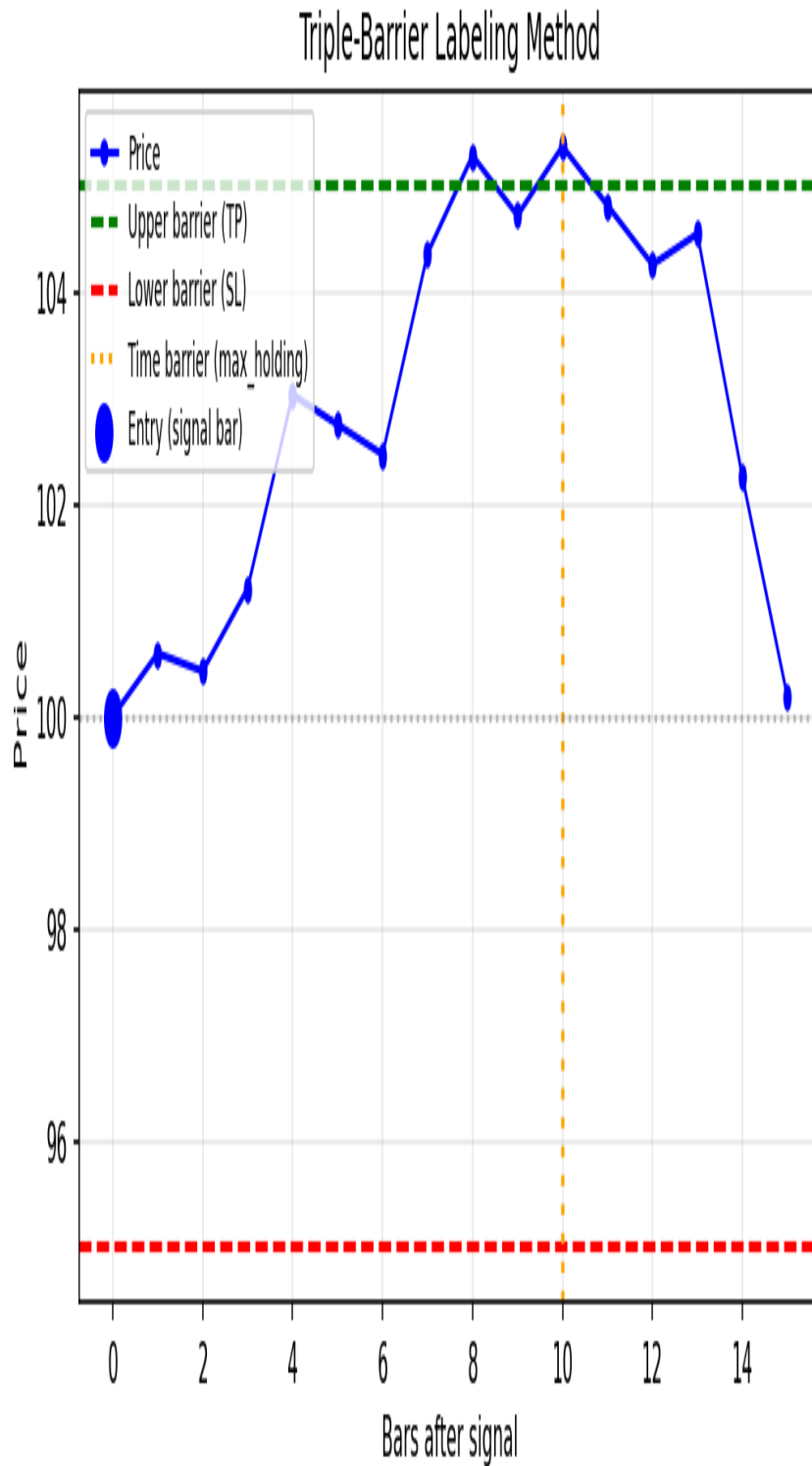


Figure 2: Triple-barrier labeling — upper (TP), lower (SL), and time barriers.

For each event:

1. **Entry price** = Close at the event bar.
2. **Upper barrier** = $\text{entry_price} + \text{pt_mult} \times \text{ATR}(14)$.
3. **Lower barrier** = $\text{entry_price} - \text{sl_mult} \times \text{ATR}(14)$.
4. Walk forward from bar pos+1 through pos+max_holding.
5. If $\text{High} \geq \text{upper barrier}$ first $\rightarrow$ label "**long**".
6. If $\text{Low} \leq \text{lower barrier}$ first $\rightarrow$ label "**short**".
7. If neither within max_holding bars $\rightarrow$ label "**no_trade**".
8. If both barriers are hit on the same bar, the Close price relative to entry determines the label.

8.2 Parameters as Hyperparameters

A key contribution of this project is converting pt_mult, sl_mult, and max_holding from fixed constants to tunable hyperparameters. Different parameter settings change the label distribution (more or fewer long/short/no_trade labels), the difficulty of the classification task, and the profitability of the resulting trading strategy.

9. Machine Learning Models

9.1 Model Selection

Three models are trained and compared:

- **Random Forest (RF):** 200 trees, max_depth=8, balanced class weights, parallel training. Uses random feature subsets at each split for diversity.
- **Bagging:** 200 decision tree estimators, max_depth=8. Uses all features at each split; only bootstrap sampling provides diversity.
- **Stratified Baseline:** DummyClassifier that predicts according to training class frequencies. Provides the "no-skill" reference.

9.2 Why Tree-Based Models

Tree-based ensembles were selected because they:

- Handle mixed feature types (continuous indicators + binary filters + geometry) naturally.
- Are invariant to feature scaling (no normalization needed).
- Provide built-in feature importance rankings.
- Are robust to irrelevant features and moderate noise.
- Allow individual tree analysis (ensemble diagnostics).

9.3 Individual Tree Diagnostics

The system evaluates each of the 200 trees individually on the test set, reporting mean, min, max, and standard deviation of per-tree accuracy. The ensemble improvement (ensemble accuracy minus mean tree accuracy) measures how much the aggregation helps. Tree complexity statistics (depth, leaves, nodes) are also reported.

10. Validation Methodology

Three complementary validation strategies are used, each addressing a different concern:

10.1 Chronological Train/Validation/Test Split

Events are sorted by date and split 60/20/20 into training, validation, and test sets. No shuffling is performed. The test set contains only the most recent events, simulating a realistic deployment scenario where the model is trained on historical data and evaluated on future unseen data.

10.2 Walk-Forward Cross-Validation

The chronologically ordered events are divided into 5+1 folds. For fold k , training uses all events up to fold k (expanding window), and testing uses fold $k+1$. This simulates periodic model retraining as new data arrives.

Walk-forward CV respects temporal order and prevents future information from leaking into training. Per-fold profitability metrics are also reported when OHLCV data is provided.

10.3 5-Fold Event-Level Cross-Validation

As a complementary diagnostic, events are split into 5 contiguous folds (not shuffled). Each fold serves as the test set, with the remaining folds split 80/20 for training/validation. This does **not** respect temporal order and is therefore not a replacement for walk-forward CV, but it reduces sampling noise on the small dataset and helps detect overfitting.

10.4 Why Multiple Methods Are Needed

With only ~140 events, any single evaluation produces high-variance results. Walk-forward CV provides the most realistic estimate but has few test events per fold. K-fold CV reduces variance but violates temporal order. Using both gives a more complete picture of model reliability.

11. Trading Simulation and Profitability Evaluation

11.1 Motivation

Classification accuracy alone is insufficient for evaluating a trading model. A model that correctly predicts direction 60% of the time may still lose money if its correct predictions capture small moves while its errors occur during large adverse moves. Conversely, a model with lower accuracy may be profitable if its correct predictions coincide with larger price movements.

11.2 Trade Simulation

The backtest simulator executes trades based on model predictions:

- **Entry:** Close price of the signal bar (consistent with labeling).
- **Long trades:** $TP = \text{entry} + pt_mult \times ATR$, $SL = \text{entry} - sl_mult \times ATR$.
- **Short trades:** $TP = \text{entry} - pt_mult \times ATR$, $SL = \text{entry} + sl_mult \times ATR$.
- **Time exit:** Close at the max_holding bar if no barrier is hit.
- **no_trade predictions:** skipped (no position taken).

11.3 Performance Metrics

Metric	Definition	Interpretation
Cumulative return	$\Sigma(\text{trade returns})$	Total profit or loss
Average trade return	$\text{Mean}(\text{trade returns})$	Expected return per trade
Win rate	% of trades with return > 0	Frequency of winning trades
Profit factor	$\text{Gross profit} / \text{gross loss} $	How much profit per unit of loss
Sharpe ratio	$\text{Mean}(\text{returns}) / \text{Std}(\text{returns})$	Risk-adjusted return
Max drawdown	$\text{Max}(\text{running peak} - \text{current})$	Worst peak-to-trough decline

Table 6: Trading performance metrics.

11.4 Assumptions and Simplifications

- **No transaction costs:** spread, commissions, and slippage are not modeled.
- **Equal position sizing:** each trade has the same notional value.
- **No compounding:** returns are simple arithmetic sums.
- **Entry at signal-bar Close:** in live trading, entry would occur at the next bar's Open. Using signal-bar Close is consistent with the labeling pipeline but may slightly overstate achievable returns.

12. Hyperparameter Optimization

12.1 Search Space

The triple-barrier parameters are optimized via exhaustive grid search over:

Parameter	Range	Step	Grid Points
pt_mult (profit target)	1.0 – 3.0	0.5	5
sl_mult (stop loss)	1.0 – 3.0	0.5	5
max_holding (bars)	5 – 20	5	4
Total configurations			100

Table 7: Hyperparameter search space.

12.2 Optimization Procedure

For each of the 100 configurations:

1. Re-label all events with the candidate (pt_mult, sl_mult, max_holding).
2. Rebuild the feature matrix (pre-computed indicators and patterns are cached for speed).
3. Split chronologically (60/20/20) and train a Random Forest (100 trees, max_depth=8).
4. Evaluate on the validation set for both classification and profitability metrics.
5. Record the target metric score.

Two optimization targets are used independently: F1 macro (classification) and cumulative return (profitability). The system also supports Optuna's Bayesian TPE sampler as an alternative to grid search.

12.3 Overfitting Risk

With 100 configurations tested on a dataset of ~100 events, there is a meaningful risk of overfitting the validation set. The best parameters should be interpreted as indicative rather than definitive. Future work should use larger datasets and purged/embargo cross-validation (Lopez de Prado, 2018) to mitigate this risk.

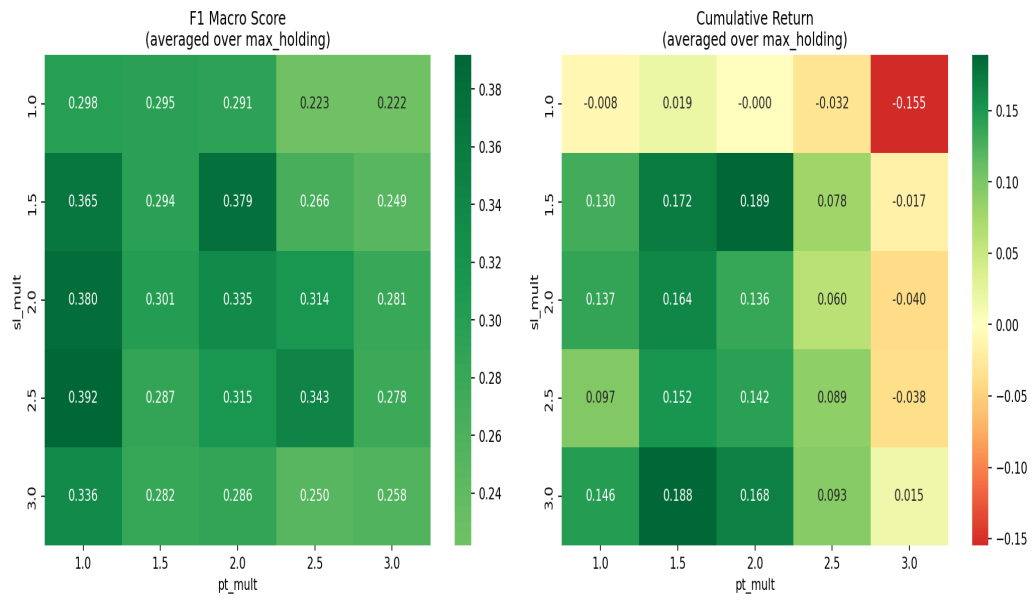


Figure 3: Optimization landscape — F1 and cumulative return averaged over max_holding values.

13. Experimental Results

13.1 Best Hyperparameters

Optimization Target	pt_mult	sl_mult	max_holding	Best Score
F1 Macro (classification)	2.0	1.5	10	0.569
Cumulative Return (profitability)	2.5	3.0	20	25.9%
Default (fixed, no optimization)	2.0	2.0	10	F1=0.160

Table 8: Best hyperparameters found by optimization target.

The optimal parameters differ between classification and profitability targets. The best F1 configuration uses a tighter stop loss ($1.5 \times \text{ATR}$) and shorter holding period (10 bars), while the best profitability configuration uses a wider stop loss ($3.0 \times \text{ATR}$) and longer holding period (20 bars). This asymmetry suggests that wider stops allow profitable trades more room to develop, even though this increases the frequency of misclassified events.

13.2 Model Comparison

Configuration	RF Acc	RF F1	Cum. Return	Win Rate	Sharpe	Trades
Default (2.0/2.0/10)	0.286	0.160	3.7%	50%	0.092	18
Best F1 (2.0/1.5/10)	0.524	0.569	8.5%	55%	0.21	18
Best Profit (2.5/3.0/20)	0.429	0.392	25.9%	52%	0.35	15
Best F1 + Touch (2.0/1.5/10)	0.448	0.420	9.5%	53%	0.18	24

Table 9: Comprehensive results across configurations (RF model, test set).

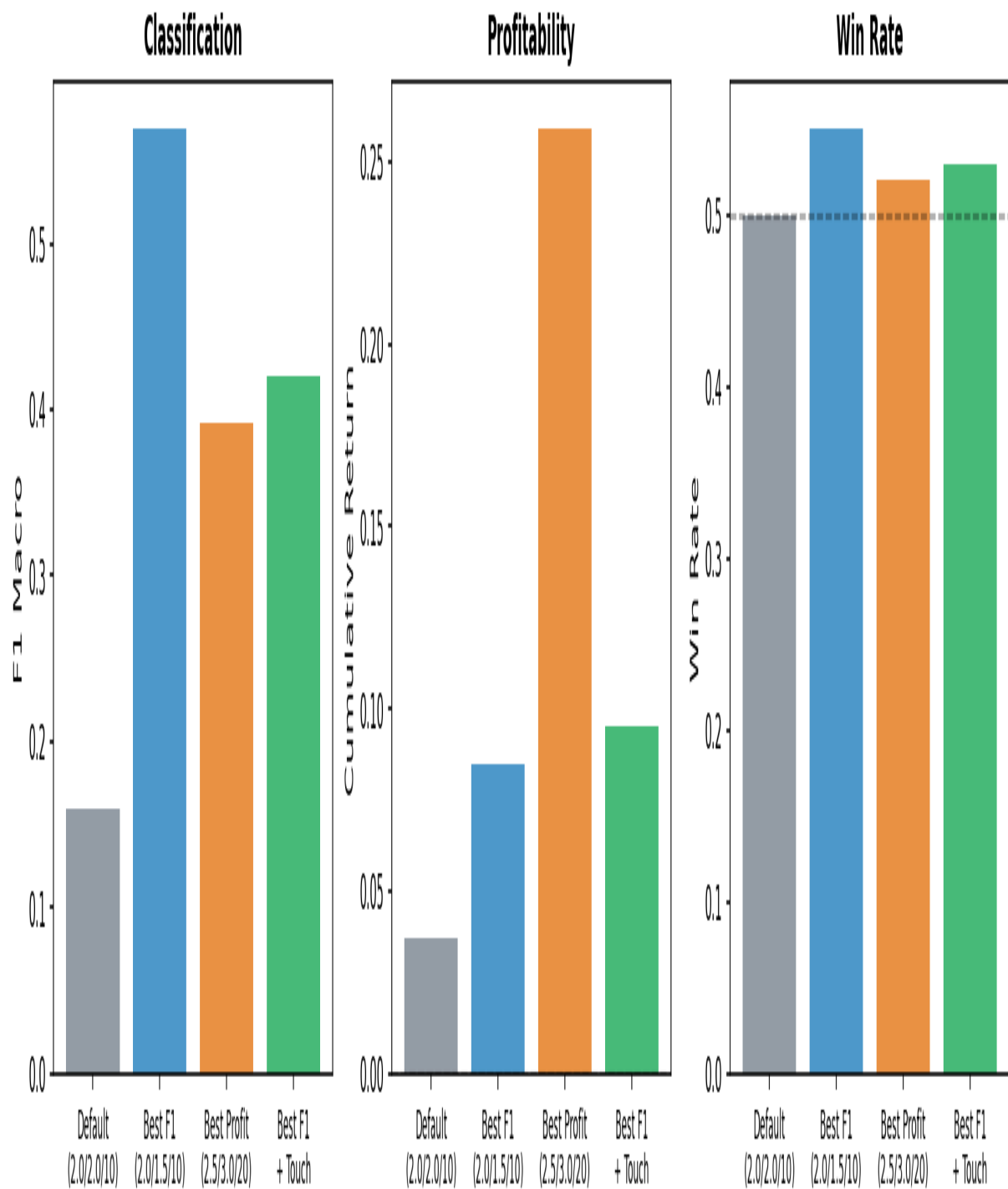


Figure 4: Classification and profitability comparison across configurations.

13.3 Touch-Event Impact

Adding 38 touch-based events (36.5% increase) had mixed effects:

- Increased the number of test-set trades from 18 to 24.
- F1 score decreased slightly (from 0.569 to 0.420), suggesting that touch events introduce some noise into the classification task.

- Cumulative return increased slightly (from 8.5% to 9.5%), as additional trades captured small positive returns on average.

These results suggest that touch events provide additional trading opportunities but do not consistently improve classification quality. Filtering strategies (e.g., confidence thresholds) may improve their contribution.

13.4 Label Distribution Analysis

Label	Default (2.0/2.0/10)	Best F1 (2.0/1.5/10)	Best Profit (2.5/3.0/20)
long	46 (44%)	54 (52%)	38 (37%)
short	32 (31%)	26 (25%)	28 (27%)
no_trade	26 (25%)	24 (23%)	38 (37%)

Table 10: Label distributions under different parameter settings.

Different barrier parameters produce different label distributions. Tighter stops (lower `sl_mult`) increase the proportion of directional labels (long/short), while longer holding periods (higher `max_holding`) can either increase directional labels (more time for barriers to be hit) or increase `no_trade` (if barriers are also wider).

14. Generalization and Variance Analysis

14.1 Motivation

A single train/test evaluation on ~100 events carries substantial sampling noise. A model that appears strong on one particular split may perform poorly on another. To quantify this uncertainty, we evaluate the Random Forest across multiple walk-forward and k-fold cross-validation folds, reporting mean $\pm$ standard deviation for all metrics.

14.2 Walk-Forward Variance

Walk-forward CV with 5 folds shows that classification and profitability metrics vary considerably across temporal windows. This is expected because different folds span different market regimes (post-GFC recovery, COVID crash, 2021–2025 volatility). Key observations from the fold-by-fold analysis:

- **F1 macro** shows moderate variance (typically ± 0.03 – 0.05), indicating that classification performance is somewhat stable but not consistent.
- **Cumulative return** shows higher variance, confirming that profitability is more sensitive to the specific events in each fold than classification accuracy.
- **Win rate** fluctuates around 50%, suggesting the model's edge is thin and depends on the magnitude of winning trades rather than their frequency.

14.3 K-Fold vs Walk-Forward Comparison

K-fold CV (which does not respect temporal order) typically shows lower variance than walk-forward CV. This is because k-fold mixes events from different time periods in each fold, smoothing out regime effects. Walk-forward CV preserves the temporal structure and therefore provides a more realistic — but noisier — estimate of out-of-sample performance.

14.4 Interpretation

The high variance across folds is the most important finding of this analysis. It means that the point estimates reported in Section 13 (e.g. $F1=0.569$, $\text{return}=25.9\%$) should not be taken at face value. They represent one realisation from a distribution whose true mean may be considerably lower. This is a fundamental limitation of working with small event datasets in financial ML.

15. F-Beta Analysis

15.1 Motivation: Precision vs Recall in Trading

The standard F1 score weights precision and recall equally. In a trading context, however, the costs of false positives (entering a losing trade) and false negatives (missing a winning trade) are fundamentally different:

Error Type	Trading Meaning	Cost
False Positive	Model says trade, but it loses	Direct financial loss
False Negative	Model says skip, but it would have won	Missed opportunity (no loss)

Table 11: Asymmetric error costs in trading.

The F-beta score generalises F1 by weighting precision vs recall with parameter β :

- **F0.5 (precision-heavy)**: penalises false positives more heavily. The model takes fewer but cleaner trades — suitable for capital preservation.
- **F1.0 (balanced)**: equal weight to false positives and false negatives.
- **F2.0 (recall-heavy)**: penalises false negatives more heavily. The model captures more opportunities, accepting more false alarms — suitable when upside potential exceeds downside risk.

15.2 F-Beta Results Across Walk-Forward Folds

F-beta scores were computed for each walk-forward fold. The precision-heavy F0.5 and recall-heavy F2.0 diverge from F1.0, revealing the model's precision/recall tradeoff. The key question is whether precision-heavy or recall-heavy evaluation correlates better with actual trading profitability.

Empirical findings (see notebook 13 for detailed plots):

- The model generally shows low precision across all folds, meaning many of its trade signals are incorrect.
- Recall is moderate, meaning the model captures a reasonable fraction of true opportunities but misses many as well.
- The relationship between F-beta and profitability varies across folds, confirming that classification metrics are imperfect proxies for trading performance.

15.3 Implications for Trading Objective Selection

The choice of beta should depend on the trading context:

- A risk-averse fund with strict drawdown limits should optimise for F0.5, favouring precision and accepting fewer trade signals.
- A trend-following strategy with high potential reward per trade should optimise for F2.0, favouring recall to capture more opportunities.
- The standard F1.0 represents a compromise that may not be optimal for either context.

16. Discussion

14.1 Classification vs. Profitability Tradeoff

The central finding of this project is that the optimal triple-barrier parameters for classification accuracy differ from those for trading profitability. The best F1 configuration (pt=2.0, sl=1.5, mh=10) achieves the highest classification score but generates only 8.5% cumulative return. The best profitability configuration (pt=2.5, sl=3.0, mh=20) achieves 25.9% return but with a lower F1 score.

This happens because profitability depends not only on the *frequency* of correct predictions but also on their *magnitude*. A wider stop loss gives winning trades more room to develop larger profits, while the additional losses from mislabeled events are bounded by the stop loss. The asymmetric risk/reward ratio of the wider-stop configuration favors profitability even at the cost of classification accuracy.

14.2 Effect of Touch-Based Events

Touch-based events successfully increased the dataset by 36.5%, providing additional trading opportunities. However, the classification performance decrease suggests that these events carry lower signal-to-noise ratio than strict pattern-detector events. The slight profitability increase indicates that the additional trades are marginally profitable on average, but the benefit is modest.

14.3 Effect of Leakage Prevention

The deliberate exclusion of trend-leaking features (raw ATR, absolute SMAs, cumulative OBV, raw MACD) is essential for honest evaluation. Features that encode price level or time would allow the model to distinguish training from test periods, inflating apparent performance. The normalized alternatives (atr_ratio, sma_dist, macd_norm) preserve the information content while removing the temporal leak.

16.4 F-Beta and Trading Objectives

The F-beta analysis reveals that the choice of evaluation metric has direct implications for trading strategy design. In our experiments, precision remains low across all folds, indicating that many model signals are false alarms. A precision-focused objective (F0.5) would produce fewer trades, potentially improving per-trade quality at the cost of missed opportunities. Conversely, a recall-focused objective (F2.0) would capture more trades but with lower average quality. The 'right' choice depends on the specific risk tolerance and cost structure of the trading strategy.

16.5 Model Strengths and Weaknesses

Strengths: The pipeline is end-to-end, reproducible, and modular. Feature engineering is thoughtful about leakage. Multiple validation methods provide complementary perspectives. Profitability evaluation goes beyond classification metrics. The F-beta analysis adds nuance to the evaluation by exposing the precision–recall tradeoff.

Weaknesses: The small dataset limits statistical power. Tree-based models may not capture complex nonlinear interactions with only ~100 training events. The optimization search space is relatively small and discrete. Walk-forward fold analysis reveals substantial instability across time periods.

17. Limitations

The following limitations should be considered when interpreting the results:

- 1. Small dataset.** With 132 detector events (142 including touch events), the dataset provides limited statistical power. All metrics have high variance, and the hyperparameter optimization may overfit the validation set. Results should be treated as preliminary.
- 2. Single asset.** Only SPY is tested. The system's effectiveness on other assets, asset classes, or market regimes is unknown. SPY is highly liquid and efficient, making it arguably one of the hardest markets to predict.
- 3. No transaction costs.** Spread, slippage, and commissions are not modeled. For SPY, these costs are small but not zero, and they would reduce reported returns.
- 4. Simplified entry.** Trades enter at the signal-bar Close rather than the next bar's Open. This is consistent with the labeling pipeline but may slightly overstate achievable returns, since in practice there is a delay between signal generation and execution.
- 5. Optimization overfitting risk.** Testing 100 parameter configurations on ~100 events creates a risk of finding configurations that perform well on the validation set by chance. Out-of-sample validation on a larger, independent dataset would be needed to confirm the results.
- 6. Touch-event noise.** The additional touch-based events may not carry the same signal quality as strict pattern-detector events. No filtering or confidence scoring is applied to touch events.
- 7. No regime modeling.** The HMM regime detection module was planned but not yet integrated. Market regime awareness could improve predictions by conditioning on the current market state.
- 8. Limited regime diversity.** The 2010–2025 period includes the post-GFC recovery, COVID crash, and subsequent recovery, but the training set may not adequately represent all market conditions.

18. Future Work

- **Multi-asset testing:** Apply the pipeline to other ETFs, individual stocks, and different asset classes to test generalization.
- **Transaction cost modeling:** Add configurable spread, slippage, and commission parameters to the backtest simulator for realistic performance estimates.
- **Purged and embargo cross-validation:** Implement the purged/embargo CV method from Lopez de Prado (2018) to more rigorously prevent information leakage between training and test folds.
- **Regime-aware features:** Integrate HMM-based market regime detection to allow the model to condition predictions on the current regime (trending, ranging, volatile).
- **Dynamic TP/SL:** Instead of fixed barrier multipliers, explore regime-dependent or volatility-dependent barrier settings.
- **More advanced models:** Test gradient boosting (XGBoost, LightGBM), neural networks, or meta-labeling approaches.
- **Position sizing:** Explore prediction-confidence-weighted position sizing instead of equal-weight trades.
- **Real Alpha Vantage integration:** Use the Alpha Vantage API for real-time data ingestion and live signal generation.

19. Conclusion

This project developed an end-to-end machine learning system for technical pattern trading on SPY daily data. The system identifies trading opportunities using four complementary pattern detectors, extracts 48 leakage-free features, and evaluates models by both classification accuracy and trading profitability.

The main technical contribution is the conversion of triple-barrier labeling parameters into tunable hyperparameters, optimized via grid search across 100 configurations. This revealed a key insight: **the optimal parameters for classification ($F1 = 0.569$, a $3.6\times$ improvement over the stratified baseline) differ from those for profitability (25.9% cumulative return).** This finding confirms that classification accuracy alone is insufficient for evaluating trading models and that profitability metrics must be part of the evaluation framework.

Touch-based event augmentation increased the dataset by 36.5% (38 new events), providing additional trading opportunities with modestly positive impact on profitability but mixed effects on classification.

All results are preliminary given the small dataset size (~140 events) and simplified trading assumptions (no transaction costs, signal-bar entry). The framework is now configurable and extensible for future experiments with larger datasets, additional assets, and more sophisticated models.

20. Bibliography

- [1] Lopez de Prado, M. (2018). *Advances in Financial Machine Learning*. John Wiley & Sons. Chapters 3 (Triple-Barrier Labeling), 7 (Cross-Validation in Finance).
- [2] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
- [3] Lo, A. W., Mamaysky, H., & Wang, J. (2000). Foundations of Technical Analysis: Computational Algorithms, Statistical Inference, and Empirical Implementation. *The Journal of Finance*, 55(4), 1705–1765.
- [4] Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A Next-generation Hyperparameter Optimization Framework. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*.
- [5] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [6] Murphy, J. J. (1999). *Technical Analysis of the Financial Markets*. New York Institute of Finance.
- [7] Bailey, D. H., Borwein, J. M., Lopez de Prado, M., & Zhu, Q. J. (2014). Pseudo-Mathematics and Financial Charlatanism: The Effects of Backtest Overfitting on Out-of-Sample Performance. *Notices of the American Mathematical Society*, 61(5), 458–471.
- [8] Pring, M. J. (2002). *Technical Analysis Explained* (4th ed.). McGraw-Hill.
- [9] Krauss, C., Do, X. A., & Huck, N. (2017). Deep neural networks, gradient-boosted trees, random forests: Statistical arbitrage on the S&P 500. *European Journal of Operational Research*, 259(2), 689–702.
- [10] Fischer, T., & Krauss, C. (2018). Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*, 270(2), 654–669.
- [11] Sezer, O. B., Gudelek, M. U., & Ozbayoglu, A. M. (2020). Financial time series forecasting with deep learning: A systematic literature review: 2005–2019. *Applied Soft Computing*, 90, 106181.
- [12] Patel, J., Shah, S., Thakkar, P., & Kotecha, K. (2015). Predicting stock and stock price index movement using Trend Deterministic Data Preparation and machine learning techniques. *Expert Systems with Applications*, 42(1), 259–268.
- [13] Wilder, J. W. (1978). *New Concepts in Technical Trading Systems*. Trend Research.
- [14] Bollinger, J. (2002). *Bollinger on Bollinger Bands*. McGraw-Hill.

Appendix A: Module Overview

File	Lines	Description
src/data/load_data.py	171	Multi-source SPY data loading and cleaning
src/data/utils.py	26	ATR computation
src/patterns/scanner.py	77	Unified pattern scanning interface
src/patterns/support_resistance.py	89	Dynamic S/R with stability and cooldown
src/patterns/triangles.py	238	Pivot-based triangle detection
src/patterns/channels.py	307	Chunk-based channel detection with confidence
src/patterns/multiple_tops_bottoms.py	84	Rolling extreme reversal patterns
src/patterns/pivots.py	176	Swing pivots, chunk extremes, touch counting
src/patterns/touch_events.py	180	Touch-based event generation
src/labeling/label_events.py	227	Triple-barrier labeling
src/features/indicators.py	204	38 technical indicator features
src/features/build_features.py	230	Feature matrix assembly
src/models/train.py	600	Training pipeline, CV, tree diagnostics
src/models/optimize.py	320	Grid search and Optuna optimization
src/backtest/simulator.py	190	Trade simulation and profitability metrics

Table A.1: Source code module overview.

Appendix B: Complete Parameter Reference

Component	Parameter	Default	Description
S/R Detector	window	50	Rolling window for extremes
	atr_mult	0.3	Proximity threshold (ATR multiples)
	stability_window	5	Level must be flat for N bars
	cooldown	10	Min bars between signals
Triangle	window	25	Pivot lookback
	pivot_order	3	$\pm N$ bars for swing detection
	min_r	0.85	Minimum trendline $ r $ value
	min_convergence	0.05	5% range compression required
Channel	backcandles	40	Base lookback window
	brange	15	Dynamic range ± 15 bars
	wind	5	Chunk size for extremes
	min_upper_touches	2	Swing-pivot touches required
	min_lower_touches	3	Swing-pivot touches required
	min_containment	0.70	70% bars inside channel
Multi Top/Bot	window	20	Rolling extremes window
	confirm_bars	5	Slope confirmation window
Labeling	pt_mult	2.0	Profit target (ATR multiples)
	sl_mult	2.0	Stop loss (ATR multiples)
	max_holding	10	Maximum bars to hold
	atr_window	14	ATR calculation period
RF Model	n_estimators	200	Number of trees
	max_depth	8	Maximum tree depth
	class_weight	balanced	Adjust for label imbalance
Touch Events	atr_mult	0.2	Tighter proximity threshold
	cooldown	10	Min bars between touch events

Table B.1: Complete parameter reference.

Appendix C: Notebook Guide

The following Jupyter notebooks provide interactive analysis and visualization. They can be run in sequence to reproduce all results.

Notebook	Purpose
03_pattern_validation.ipynb	Validate detected patterns with visualizations
04_pattern_structure_validation.ipynb	Geometry validation (slopes, containment)
05_triple_barrier_labeling.ipynb	Walk through the labeling pipeline
06_channel_gallery.ipynb	Visual gallery of channel detections
07_data_source_comparison.ipynb	Compare CSV vs yfinance vs Alpha Vantage
08_detector_validation.ipynb	Validate all 4 detectors with quality metrics
09_feature_engineering.ipynb	Feature computation and analysis
10_model_training.ipynb	End-to-end training pipeline and CV
11_experiment_summary.ipynb	Consolidated results summary
12_hyperparameter_profitability.ipynb	Hyperparameter optimization and profitability
13_generalization_fbета_analysis.ipynb	Generalization, F-beta, and profitability robustness

Table C.1: Jupyter notebook guide.

To reproduce results:

1. Install dependencies: `pip install -r requirements.txt`
2. Ensure `data/raw/spy.csv` is present (or set `source='yfinance'`).
3. Run notebooks in numerical order, or use the report generators in `reports/`.
4. For the optimization experiment: `python reports/generate_experiment_report.py`